Project 3: SSD Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We profile a Micron 2210 512 GB QLC NVMe SSD on a PCIe 3.0 ×4 link with `O_DIRECT` I/O against disposable files, using a `libaio` benchmark as the primary engine and FIO as a cross-check. At QD=1, 4 KiB random reads take 53.2 µs (18 783 IOPS) and writes 16.0 µs, because the write buffer hides the flash program. With one request in flight, sequential reads level off near 1 444 MiB/s. That limit comes from per-request latency, not the interface. At QD=32 they top out at 2 144 MiB/s, only 57 % of the 3756 MiB/s raw PCIe link ceiling, so the NAND back end sets the limit. Random-read IOPS saturate at QD 64 (149 422 IOPS) under a stated knee criterion, and Little's Law serves as a consistency check on the closed-loop harness. Writes land in an SLC cache at 1 070 MiB/s. Once it is full, sequential writes fall to 62 MiB/s and 4 KiB random writes to 9 622 IOPS. Reads issued while the drive is still folding recent writes show p99.9 latencies up to 24× their median, and sequential reads of randomly written data run 6–9× slower. The required misleading setup, dropping `O_DIRECT`, over-reports throughput 19× because it measures the page cache.

1 Introduction

Modern NVMe SSDs deliver high IOPS and throughput only when requests are issued concurrently. Like memory, storage shows a queueing-driven throughput–latency tradeoff: deeper queues raise device utilization up to a saturation knee, and past it latency climbs sharply for little extra throughput. This project characterizes that behaviour on a Micron 2210 512 GB QLC NVMe SSD. It measures QD=1 latency, block-size and read/write-mix effects, the queue-depth knee, tail latency and device state (working set, SLC-cache exhaustion, temperature). All I/O is `O_DIRECT` against disposable files. The primary engine is a small `libaio` benchmark, and FIO cross-checks it (Sections 2 and 3).

This report answers:

1. What are the QD=1 latencies for small random and large sequential I/O? (Section 4)
2. How do block size, access pattern, read/write mix and queue depth shape IOPS, MiB/s and latency, and where is the PCIe interface ceiling? (Sections 5 and 6)
3. Where is the queue-depth knee, by what criterion, and what does Little's Law say about the harness? (Section 7)
4. What do tail latency and device state reveal: working-set size, SLC-cache exhaustion and steady state, data layout, and temperature? (Sections 8 and 9)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	a single I/O-issuing thread, not CPU-pinned (storage I/O is device-bound, not CPU-bound, below the host-saturation queue depths of Section 7); no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
SSD	Micron 2210 `MTFDHBA512QFD`, 512 GB QLC NVMe, firmware `P6MA001` (from `/sys/class/nvme/nvme0`; SMART logs need root)
Interface	PCIe link 8.0 GT/s ×4 (Gen3), both current and maximum, read from `current_link_speed`/`current_link_width` in sysfs. Raw ceiling $8\,\mathrm{GT/s} \times 4 \times \frac{128}{130} = 3.94\,\mathrm{GB/s}$ (3756 MiB/s), roughly 3.2 GB/s to 3.5 GB/s after TLP/DLLP overhead
Block layer	scheduler `none`, `nr_requests` 1023, 16 I/O queues, volatile write cache reported as "write back"
Target	disposable files on the SSD's ext4 `/` partition (`nvme0n1p3`, mounted `relatime`, no online discard), reached via `ACS_SCRATCH`: a 2 GiB test file for the sweeps, and a 60 GiB file for the device-state runs
Temperature	NVMe composite sensor `/sys/block/nvme0n1/device/hwmon*/temp1_input`, readable without root, logged per repetition and every second in the sustained runs
Env. snapshot	2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`)

5. Which setup mistake produces misleading numbers? (Section 10)

In summary, QD=1 random reads cost 53.2 µs and writes only 16.0 µs, because the write buffer absorbs them. With one request in flight, sequential reads are limited by per-request latency rather than the PCIe 3.0 ×4 link. Deeper queues raise them to only 57 % of the link, where the NAND back end becomes the limit (Section 5). The random-read knee sits at QD 64. Tails stretch with queueing and with background flash work. Once the SLC cache is full, sustained writes drop 17× to the direct-to-QLC rate, and the drive's recent write history changes both read tails and sequential-read speed (Section 9). Buffered I/O over-reports throughput by more than an order of magnitude because it measures the page cache, not the device.

2 Methodology and Safety

2.1 Platform

The machine has two drives: this Micron NVMe SSD and a 2 TB SATA HDD (`/dev/sda`, mounted at `/data`). The scratch directory is placed on the SSD's filesystem. Each run's `env.txt` resolves the scratch filesystem's partition to its parent disk (`nvme0n1p3`→`nvme0n1`) and records that disk's `rotational` flag (0), model, firmware, PCIe link and temperature, so a run can never be silently mislabelled (Section 12 describes how this check once failed).

2.2 Data-loss safeguards

- **Targets.** Only disposable files (`testfile.bin`, `bigfile.bin`) in a dedicated scratch directory are ever written; nothing else on the SSD is touched. Both files are deleted when `run.sh` exits, including on interrupt.
- **Gate.** `scripts/safety.sh` runs before every experiment. It hard-refuses a block or character device (including through a symlink), a mountpoint, a directory, any path outside the scratch directory, and a target without enough free space. `scripts/validate.sh` (`make test`) verifies that it refuses `/dev/nvme0n1` and `/dev/sda`, a symlink to `/dev/sda`, `/etc/passwd` and `/`. These checks only `stat` the paths and never open them.
- **Benchmark.** `blkbench` `lstat`s its target *before* opening it, refuses anything that is not a regular file, and opens with `O_NOFOLLOW`. No raw device or partition is ever opened.
- **Space.** The large device-state file (60 GiB here) is refused unless the filesystem keeps at least 10 GiB free after creating it.

2.3 I/O engines and configuration

The primary engine is `build/blkbench`, a small O_DIRECT benchmark built on Linux `libaio`, the same asynchronous engine as FIO's `libaio` ioengine [1]. It keeps exactly QD requests in flight in a closed loop: every completion is immediately replaced. It records the submit-to-completion latency of every I/O. FIO 3.42 runs the same points through `fio/job.fio` (`libaio`, `direct=1`, total-latency percentiles), and Section 3 compares the two engines. `ENGINE=fio` routes every standard sweep through FIO instead.

Engine	`libaio`; `O_DIRECT` (page cache bypassed); 4 KiB-aligned buffers and block-aligned offsets; 64-bit uniform random offsets (splitmix64)
Timing	4 s per repetition (6 s for tail runs). Points that write also stop after 256 MiB per repetition, which bounds the bytes written (Table 2). No separate ramp: preconditioning absorbs warm-up
Repetitions	5 per point (3 for the block-size write points, also to bound bytes written); figures and tables show the median with inter-quartile-range (IQR) bars
Data pattern	incompressible: every write buffer is filled once, before timing, from an xorshift64* generator seeded per queue slot; the preconditioning pass uses the same generator. `make test` checks that `gzip` cannot compress the result
Latency metric	per-I/O submit→completion latency; p50/p95/p99/p99.9 from the full sorted sample, with no reservoir truncation
Preconditioning	one sequential O_DIRECT write pass over the `fallocate`d test file, repeated before every *sequential-read* group, because random writes scatter the physical layout (Section 9). The current `run.sh` also preconditions before *every* experiment, including inside `all`. In the reported 2026-10-21 run, `all` preconditioned only once, before `zero_queue`. Later experiments started from the file the previous one left; only the sequential-read groups of `zero_queue`, `block_size`, `pitfall` and `layout` were re-filled. `rerun_clean.sh` (Section A) preconditions before each experiment
Temperature	NVMe composite temperature recorded with every repetition (`temp_c`)
Driver	`SSD_Performance_Profiling/scripts/run.sh`; benchmark `src/blkbench.c`

Because the issuing thread only submits and reaps completions, one thread can keep the device's queue full. The Little's-Law check in Section 7 shows where this stops being true.

Table 2: Measured wall-clock time of each experiment in the reported run, in execution order, and the maximum bytes it can write with the default knobs (byte caps plus preconditioning).

Experiment	Time	Max written	Notes
`zero_queue`	1.1 min	7 GiB	includes the initial preconditioning
`block_size`	9.1 min	22 GiB	48 points; write points 3 reps, 256 MiB cap
`rw_mix`	0.5 min	4 GiB	
`queue_depth`	3.7 min	11 GiB	read and write, QD 1–256
`tail`	2.0 min	–	reads only
`pitfall`	0.8 min	2 GiB	reads only (plus re-fill)
`layout`	2.2 min	6 GiB	includes a 4 GiB random overwrite
`fio_check`	3.3 min	8 GiB	both engines, interleaved
`sustained`	24.9 min	72 GiB	60 GiB fill + ≤8 GiB random + ≤4 GiB recovery
`working_set`	4.0 min	5 GiB	on the 60 GiB file left by `sustained`

2.4 Execution and run order

The timed runs were executed by the author in a terminal, using the scripts described here, with the CPU governor at `performance`, turbo off and SMT off. `run.sh all` ran the experiments in the order of Table 2 on 2026-10-21, from 10:32 to 10:54. `SUSTAIN_GIB=60 run.sh device_state` followed immediately, from 10:54 to 11:23. Harness development and a discarded pilot run took place from 2026-10-12 to 2026-10-20; all reported numbers come from the 2026-10-21 runs. The drive's state carries over from one experiment to the next, and nothing in between trims or idles it: the SLC-cache fill level and the background folding/GC backlog are both inherited. Several results below depend on this, and each says so: the write collapse in Section 5, the tail runs in Section 8, and the whole of Section 9.

3 Validation and Sanity Checks

Before trusting any number, we confirm that the benchmark describes the SSD and not an artifact. `make -C SSD_Performance_Profiling test` (`scripts/validate.sh`) runs the automated checks; it writes about 70 MiB and deletes it.

- **Safety gate.** `safety.sh` refuses both block devices on the host (`/dev/nvme0n1`, the SSD; `/dev/sda`, the HDD), a symlink to `/dev/sda` placed inside the scratch directory, a directory, a path outside the scratch directory (`/etc/passwd`) and a mountpoint (`/`). It accepts only a disposable file inside the scratch directory. `blkbench` independently refuses a directory and a symlink before opening them. None of these checks opens a device.

- **Incompressible data.** The preconditioned file does not shrink under `gzip -1` (output $\geq 99\%$ of input), so the drive cannot gain from compression or zero detection.

- **Physical orderings.** On a preconditioned file, 128 KiB sequential throughput exceeds 4 KiB random throughput. Raising QD from 1 to 8 raises the measured concurrency $L = \lambda W$, and L never exceeds the configured QD (a hard invariant of a closed loop with QD slots; Section 7).

- **Direct I/O is real.** With `O_DIRECT` the QD=1 4 KiB reads take tens of microseconds. The `-buffered` path reaches about two million IOPS at sub-microsecond latency because the page cache serves it from RAM (Section 10). A gap of two orders of magnitude shows that the real

Table 3: `blkbench` vs. FIO on identical points (medians of 5 interleaved repetitions each; latencies in µs). Data: 2026-10-21 run.

Pattern	Block	QD	IOPS		FIO/blk	p50		p99	
			blk	FIO		blk	FIO	blk	FIO
rand read	4 KiB	1	18 763	18 438	0.983	49.9	50.4	58.7	57.6
rand read	4 KiB	32	129 580	129 705	1.001	217.1	216.1	663.5	692.2
seq read	128 KiB	1	1 793	1 804	1.006	546.4	544.8	1 057.9	1 036.3
seq read	128 KiB	32	1 827	1 827	1.000	17 401.3	17 432.6	19 449.4	19 791.9
rand write	4 KiB	1	59 931	55 681	0.929	14.7	15.4	17.7	19.1
rand write	4 KiB	32	236 169	184 608	0.782	121.1	117.2	165.0	618.5
seq write	128 KiB	1	8 132	8 569	1.054	100.0	98.8	335.4	280.6

runs bypass the cache. `make test` asserts that buffered re-reads are more than 5× faster than direct ones.

- **Interface ceiling known, not assumed.** sysfs reports the link at 8 GT/s ×4, so no measurement may exceed 3756 MiB/s (Section 2). The highest sequential read we measure, 2 144 MiB/s (64 KiB, QD=32) (Section 5), is checked against this bound.
- **Device identity.** Every run's `env.txt` records the parent disk of the scratch filesystem, its `rotational` flag (0), the model string `Micron_2210_MTFDHBA512QFD`, the firmware and the PCIe link.

`iostat` and `perf` are not available here, so the effective queue depth is checked in-band instead (Table 6).

3.1 Cross-check against FIO

`run.sh fio_check` runs seven points (QD=1 and QD=32; reads and writes; 4 KiB random and 128 KiB sequential) through both engines on the same preconditioned file. The engines alternate repetition by repetition, so drift affects both equally. FIO uses `fio/job.fio`: `libaio`, `direct=1`, one request per submit, total-latency (`lat`) percentiles, which is the same submit→completion quantity `blkbench` records. Table 3 compares the medians. We treat agreement within 10 % in IOPS and p50 as validating `blkbench`. FIO's percentiles come from a log-bucketed histogram (at most ∼3 % bin error), so small p99 differences are expected.

Reads agree. On all four read points, FIO/`blkbench` IOPS ratios are 0.983–1.006 and p50 latencies differ by at most 1 %, so `blkbench` is validated for reads. The two 128 KiB sequential read points ran at only ∼225 MiB/s in *both* engines. In `run.sh all`, `fio_check` directly followed `layout`, which leaves the file randomly overwritten, and the script did not rewrite it in between (since fixed). These points therefore measured the fragmented layout of Section 9.3. That does not affect the engine comparison, but their absolute values must not be read as sequential-read performance.

Writes partly disagree. QD=1 sequential writes agree (ratio 1.054) and QD=1 random writes are within the criterion (0.929, p50 14.7 vs. 15.4 µs). The QD=32 random-write point fails it: FIO reached 184 608 IOPS against `blkbench`'s 236 169 (0.78), with p99 at 619 vs. 165 µs. The medians, by contrast, agree (117 vs. 121 µs). Identical medians argue against a per-I/O engine cost. The deficit sits in the tail, which is the signature of the drive pausing to flush its write buffer. Within each pair of repetitions, `blkbench` always ran first and FIO second, immediately after `blkbench`

Table 4: QD=1 performance on the Micron 2210 NVMe SSD: medians over repetitions, with the IQR in brackets for IOPS and p99; latencies in µs. Random reads pay the full flash-read latency; random writes are faster because the write buffer absorbs them.

Pattern	Block	IOPS [IQR]	MiB/s	Mean	p50	p95	p99 [IQR]	p99.9
rand read	4 KiB	18 783 [18 767, 18 796]	73.4	53.2	49.9	50.9	62 [62, 71]	534
rand write	4 KiB	62 217 [62 067, 62 239]	243.0	16.0	14.6	15.9	18 [18, 18]	490
seq read	128 KiB	11 530 [11 511, 11 543]	1 441.2	86.7	80.5	114.2	491 [490, 496]	563
seq write	128 KiB	5 100 [891, 5 298]	637.5	196.0	90.0	1 156.4	1 437 [1 436, 34 765]	9 853

had written 256 MiB. FIO therefore met a fuller buffer more often. We consider this ordering effect the most likely explanation, but we did not test it: that would need the engine order alternated or randomized per pair. Until then, write numbers from either engine carry an uncertainty of about 20 % at QD=32, set by drive state rather than by the tool. The read results, which are not order-sensitive, show no such effect.

Engine order. The reported run always ran `blkbench` first. `fio_check` now alternates the order per repetition and records each row's position (`fio_order.csv`), so the next run can separate drive state from engine.

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

At QD=1 a single I/O is in flight, so latency is the full host-stack + controller + flash path. We expect 4 KiB random reads in the tens of microseconds (flash read dominates) and 128 KiB sequential transfers to be bandwidth-limited at a few hundred µs. Writes should be *faster* than reads at QD=1 because the SSD's DRAM/SLC write buffer acknowledges them before the flash program completes.

Fixed	iodepth 1; one job; `O_DIRECT`; 5 repetitions of 4 s (write points also stop at 256 MiB)
Cases	4 KiB random read, 4 KiB random write, 128 KiB sequential read, 128 KiB sequential write
Order	random read → random write → sequential re-fill of the file → sequential read → sequential write (the re-fill keeps the sequential read independent of the random writes; Section 9.3)
Script	`SSD_Performance_Profiling/scripts/run.sh zero_queue`

4.1 Analysis

The QD=1 numbers identify an NVMe SSD, not a rotational disk. 4 KiB random read latency is 53.2 µs, not the milliseconds an HDD would show, and it is dominated by the NAND read plus the NVMe/`libaio` host path, with no seek. Random *writes* take only 16.0 µs (62 217 IOPS), because the controller acknowledges them from its write buffer (SLC cache plus controller SRAM) before the flash program finishes. A QD=1 read has no such buffer to hide behind.

The 128 KiB sequential rows are limited by per-request latency. The read takes 87 µs per request (1 441 MiB/s), and the single outstanding request leaves the device idle between transfers. Sections 5 and 7 show throughput climbing once larger blocks or deeper queues keep more data moving.

Two caveats apply. First, a QD=1 random read cannot be prefetched, so this is a genuine floor. Second, the p99.9 column already shows occasional outliers of several hundred microseconds to

milliseconds from background flush and garbage collection, which preview the tail behaviour in Section 8.

The sequential-write row is unstable. Its IOPS IQR spans 891–5 298, and its p99 IQR runs to 34 764.7 µs. The raw repetitions are bimodal. The first two ran at 490 and 891 IOPS with p99 of 30 ms to 35 ms; the last three ran at 5100 to 5460 IOPS with p99 of 1.4 ms. The p50 stays near 90 µs in all five, so typical writes were fast throughout, and the slow repetitions are dominated by a few stalls of tens of milliseconds. The point ran directly after the 2 GiB sequential re-fill that precedes the sequential-read case. We read the first two repetitions as waiting for the controller to flush that burst, and the later ones as the unloaded QD=1 write path. The median (5 100 IOPS) therefore describes the recovered state, and the row should not be read as a stable baseline. `rerun_clean.sh` repeats it after a rest, and `blkbench` now logs each point's start time (`timeline.csv`), so such stalls can be placed in time.

An early pilot run, since discarded, measured this sequential read directly after the random-write case and got only a fraction of the rate above. That discrepancy led to the re-fill step in the procedure, and the `layout` experiment (Section 9.3) reproduces it: 224 vs. 1 443 MiB/s.

5 Block-Size Sweep

Hypothesis (from first principles, then compared with the data)

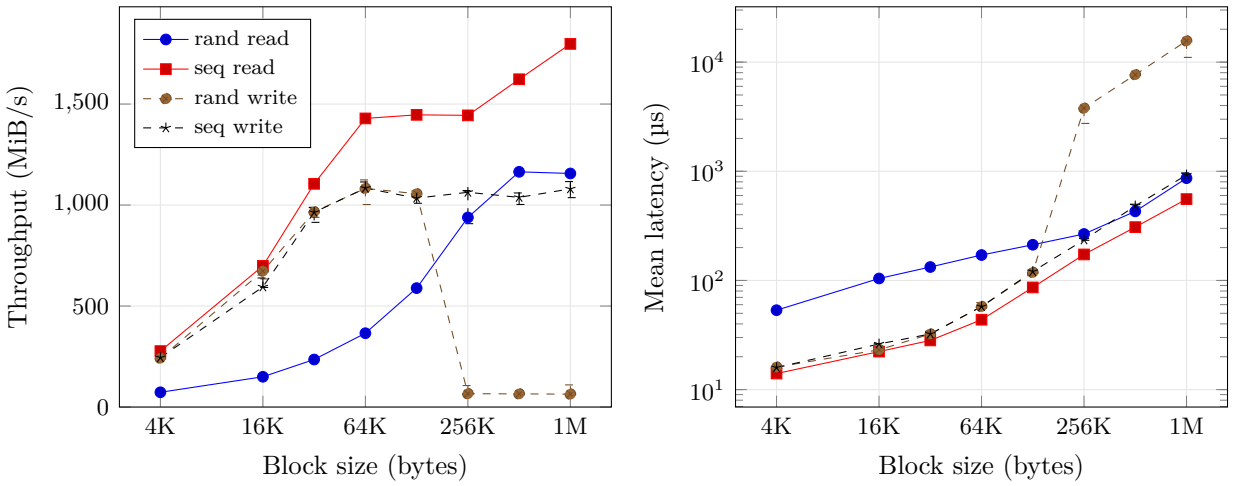
As the block grows from 4 KiB to 256 KiB at QD=1, small-block I/O should be *IOPS-limited* (throughput = IOPS × block, rising with block size) and large-block I/O should approach a *throughput ceiling* where IOPS falls as 1/block. Sequential should beat random at every block size, with the gap largest at small blocks where per-op overhead dominates.

Fixed	one job; `O_DIRECT`; 5 repetitions of 4 s for reads; 3 repetitions capped at 256 MiB each for writes
Varied	block size 4 KiB–1 MiB (8 sizes); random vs. sequential; 100% read vs. 100% write; QD=1 (all four combinations) and QD=32 (sequential read and write)
Ceiling	PCIe 3.0 ×4: 3756 MiB/s raw (Section 2)
Script	`SSD_Performance_Profiling/scripts/run.sh block_size`

5.1 Analysis

Random reads at QD=1 are latency-bound, not size-bound. IOPS fall only from 18 711 at 4 KiB to 3 754 at 256 KiB, so throughput climbs from 73 to 939 MiB/s. Per-request latency is not fixed: it grows from 53 to 266 µs (5×) over a 64× range of sizes. A useful decomposition is $W(b) \approx W_0 + t_{\text{xfer}}(b)$: a fixed access cost W_0 of roughly 50 µs (NAND sense plus command overhead, visible at 4 KiB) plus a transfer term. Up to 256 KiB the transfer term grows *sub-linearly* with b, because the controller stripes a request across several dies and planes and reads them in parallel. Beyond that the stripes are exhausted. Latency then doubles with size (429 µs at 512 KiB, 865 µs at 1 MiB), and throughput plateaus at 1 164–1 156 MiB/s.

Sequential reads at QD=1 hit a single-request limit. The 4 KiB sequential read takes only 14.0 µs, versus 53.4 for random. The controller detects the stream and reads ahead, so most requests hit its internal buffer. From 64 KiB to 256 KiB, throughput is flat at 1 429–1 447 MiB/s, and IOPS fall as $1/b$. Larger requests then lift it to 1 798 MiB/s at 1 MiB, because each command covers more of the internal pipeline. Even that is only 48 % of the 3756 MiB/s PCIe 3.0 ×4 link. The QD=1 limit is therefore how fast the controller streams one command at a time, not the interface. The sequential/random gap is widest at 4 KiB and narrows as both patterns become transfer-dominated.



(a) QD=1 throughput.

(b) QD=1 mean latency (same runs).

Figure 1: QD=1 block-size sweep, reads (solid) and writes (dashed); medians with IQR bars. Random-read throughput rises with block size because latency grows more slowly than the transfer size. Sequential reads stay near 1.4 GiB/s from 64 to 256 KiB and reach 1.8 GiB/s at 1 MiB, a single-request limit well below the 3756 MiB/s link. The random-write collapse at ≥256 KiB is an SLC-cache-exhaustion episode (see text). Data: 2026-10-21 run.

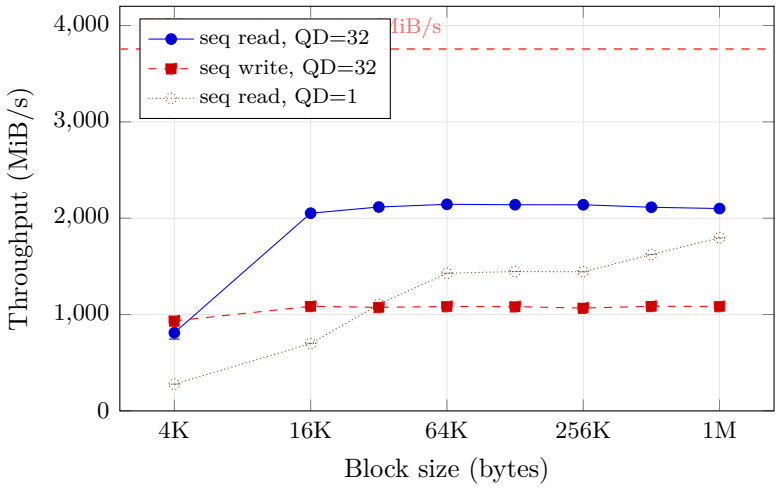


Figure 2: Sequential throughput at QD=32 against the measured-link ceiling (red), with the QD=1 read curve for reference. Deep queues remove the per-request latency limit, so the remaining gap to the line is the device's internal bandwidth plus protocol overhead. Data: 2026-10-21 run.

Table 5: Read/write mix at QD=16, 4 KiB random: medians with the IQR in brackets; latencies in µs.

Mix	Total IOPS	Mean latency	p99
100% read	102 514 [102 472, 102 633]	154.1 [153.9, 154.2]	556 [555, 561]
70/30	81 497 [79 349, 84 712]	193.7 [186.4, 199.3]	757 [718, 796]
50/50	69 172 [68 792, 72 950]	229.4 [217.3, 230.8]	989 [962, 1 008]
100% write	253 874 [216 774, 254 678]	62.7 [62.5, 72.8]	79 [69, 528]

Deep queues do not reach the link ceiling (Figure 2). At QD=32, sequential reads flatten at 2 052–2 144 MiB/s from 16 KiB upward. They reach 2 100 MiB/s at 1 MiB, about 56 % of the raw link rate and well below the 85 % to 90 % that TLP/DLLP overhead would allow. The binding limit is the drive's NAND/controller back end, not PCIe. Sequential writes at QD=32 are flat at 1 084 MiB/s for every block from 16 KiB up. That is the SLC-cache ingest rate: the same ∼1.06 GiB/s appears in the preconditioning passes and in the first seconds of the sustained fill (Section 9).

Writes at QD=1. At 4 KiB, random writes post 61 897 IOPS against 18 711 for reads. The write buffer acknowledges before the NAND program, so write latency reflects the host→buffer transfer, not flash timing. Accordingly, the random and sequential write curves coincide up to 128 KiB (1 083 vs. 1 084 MiB/s at 64 KiB). Both flatten at about 1.05 GiB/s from 64 KiB, the same ingest rate as at QD=32. Unlike reads, writes reach their ceiling with a single request in flight.

An SLC-exhaustion episode inside the sweep. The random-write points at 256 KiB, 512 KiB and 1 MiB fell to 66–64 MiB/s, with p99 latencies of 33 ms to 45 ms. That is the same ∼60 MiB/s direct-to-QLC rate the sustained experiment measures once the SLC cache is full. These were the last write points before the sequential-read group, and the run had written about 10 GiB by then (preconditioning, `zero_queue` and the smaller write points). The sequential writes, measured a few minutes later after the read-only group, were back at ∼1.05 GiB/s. This is consistent with the controller having drained part of the cache in between. Treat the three collapsed points as a drive-state measurement, not a block-size effect: the sequential-write curve shows no such drop at the same sizes.

6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

At a fixed block size and queue depth, a mixed read/write workload should deliver *fewer* total IOPS than either pure workload, not a weighted average: reads and writes contend for the controller and flash channels, and a read queued behind a slow program/erase inherits its latency. We expect pure writes to post the highest IOPS at QD=16 (write-buffer absorption) and the 50/50 mix to be the worst.

Fixed	4 KiB random; iodepth 16; one job; `O_DIRECT`; 5 repetitions of 4 s (points with writes also stop at 256 MiB)
Varied	reads per 10 I/Os, deterministically interleaved: 10 (100% R), 7 (70/30), 5 (50/50), 0 (100% W)
Script	`SSD_Performance_Profiling/scripts/run.sh rw_mix`

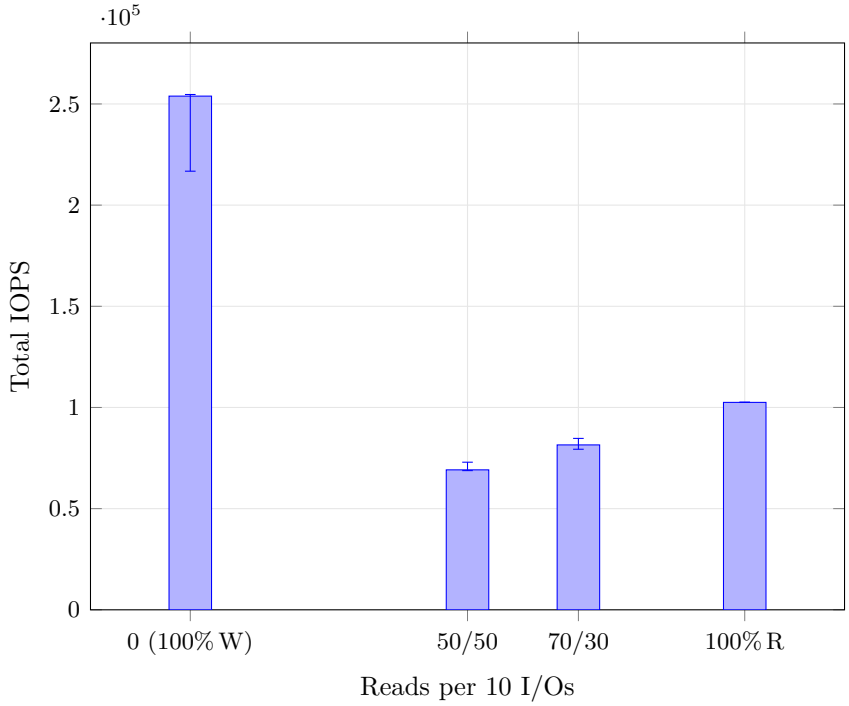


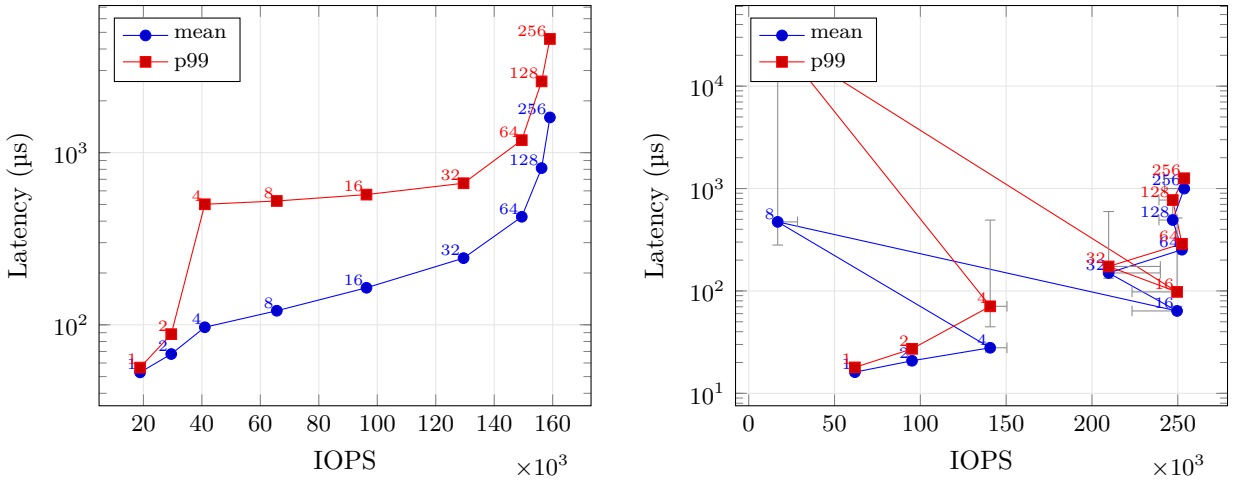
Figure 3: Total 4 KiB random IOPS at QD=16 across read/write mixes (median, IQR bars). Both pure workloads beat the mixes; the 50/50 mix is the worst.

6.1 Analysis

Pure writes are fastest, at 253 874 IOPS and 63 µs mean, because at QD=16 the controller absorbs them into its SLC/SRAM buffer. Pure reads reach 102 514 IOPS. Mixing is worse than either pure case. The 70/30 mix drops to 81 497 IOPS and the 50/50 mix to 69 172, below even the read-only number. A write landing mid-stream occupies a die (program, or the buffer flush it triggers), and reads mapped to that die queue behind it. The interleaved writes also break up the write buffer's batching.

The p99 latency gives a related but not monotonic picture. It rises from 556 µs for pure reads to 757 (70/30) and 989 µs (50/50). Pure writes break the trend: their median p99 of 79 µs is the lowest of all four mixes, but its IQR (69–528 µs) is by far the widest. A pure-write repetition either stays inside the write buffer and sees no tail, or it meets a buffer flush and sees a read-like tail. The worst *typical* tail belongs to the 50/50 mix, not to the largest write fraction, so p99 is not monotonic in the write fraction.

The lesson: a "70% read" workload is not 0.7 of the read result plus 0.3 of the write result. The interaction term is large and negative, which is why the spec requires measuring the mixes rather than interpolating them.



(a) 4 KiB random read.

(b) 4 KiB random write.

Figure 4: Throughput vs. latency as queue depth grows; each point is labelled with its QD. Bars show the IQR across repetitions in both IOPS and latency. Read IOPS reach 149 422 at the knee (QD 64) and at most 159 047 at QD=256; deeper queues then add only latency. Data: 2026-10-21 run.

7 Queue-Depth Sweep

Hypothesis (from first principles, then compared with the data)

IOPS should rise steeply with queue depth as the SSD's parallel NAND channels fill, then saturate at a knee beyond which latency grows roughly linearly with queue depth for no extra throughput. By Little's Law the number of outstanding I/Os $L = \lambda W$ should track the configured iodepth while the queue stays full; where it stops tracking reveals a host-side or engine limit.

Fixed	4 KiB random; one job; `O_DIRECT`; 5 repetitions of 4 s (writes also stop at 256 MiB)
Varied	iodepth 1, 2, 4, …, 256; 100% read and 100% write
Knee rule	the smallest QD whose doubling raises median IOPS by less than 10 % (computed by `run.sh` into `queue_depth_knee.csv`)
Effective QD	checked in-band with Little's Law (Table 6); no `iostat` available
Script	`SSD_Performance_Profiling/scripts/run.sh queue_depth`

7.1 Knee

We define the knee as *the smallest QD whose doubling raises median IOPS by less than* 10 %. Past that point, most of any extra concurrency becomes added latency rather than throughput.

Reads. The rule gives QD 64 (149 422 IOPS, 424 µs mean). Doubling to QD 128 adds only 4.5 % IOPS. The preceding doubling, QD 32→64, still gained 15 %. The grid doubles at each step, so the knee lies between 32 and 64.

Writes. Applied literally, the rule returns QD 4. That is an artifact. The QD=8 write point caught a buffer-flush stall: median 16 884 IOPS with an IQR of 16 202–28 437 and a p99 of 27 885.3 µs. The rule therefore saw a fall rather than a gain. Read with that point excluded, write IOPS rise from 61 860 (QD=1) to 140 698 (QD=4) and 249 588 (QD=16), then stay at 209 717–253 743 out to QD=256. The write knee is effectively QD ≈ 16. The write plateau of ∼250k IOPS × 4 KiB ≈

Table 6: Little's Law per queue depth: λ (IOPS) and W (mean latency, µs) are medians with the IQR in brackets; $L = \lambda W$ is computed per repetition and then summarized. `4 KiB` random.

Mode	QD	λ (IOPS) [IQR]	W [IQR]	$L = \lambda W$ [IQR]	L/QD
read	1	18 866 [18 854, 18 885]	52.9 [52.9, 53.0]	1.00 [1.00, 1.00]	0.998
read	2	29 518 [29 496, 29 525]	67.6 [67.6, 67.7]	2.00 [2.00, 2.00]	0.998
read	4	41 006 [40 991, 41 072]	96.8 [96.6, 96.8]	3.97 [3.97, 3.97]	0.992
read	8	65 629 [64 935, 65 709]	120.7 [120.5, 122.0]	7.92 [7.92, 7.92]	0.99
read	16	96 303 [96 269, 96 406]	164.1 [163.9, 164.2]	15.80 [15.80, 15.80]	0.988
read	32	129 513 [129 050, 129 607]	243.9 [243.8, 244.4]	31.59 [31.54, 31.60]	0.987
read	64	149 422 [149 292, 149 458]	424.2 [424.2, 424.5]	63.38 [63.38, 63.39]	0.99
read	128	156 211 [156 149, 156 431]	813.4 [812.2, 813.6]	127.05 [127.04, 127.06]	0.993
read	256	159 047 [158 980, 159 094]	1 602.6 [1 602.2, 1 603.3]	254.89 [254.89, 254.89]	0.996
write	1	61 860 [61 800, 62 223]	16.1 [16.0, 16.1]	1.00 [1.00, 1.00]	0.996
write	2	95 161 [92 914, 95 226]	20.8 [20.8, 21.3]	1.98 [1.98, 1.98]	0.989
write	4	140 698 [138 120, 150 450]	27.9 [26.1, 28.4]	3.93 [3.92, 3.93]	0.982
write	8	16 884 [16 202, 28 437]	472.5 [280.6, 492.4]	7.98 [7.98, 7.98]	0.997
write	16	249 588 [223 357, 250 147]	63.8 [63.6, 70.8]	15.91 [15.81, 15.92]	0.994
write	32	209 717 [209 279, 239 844]	149.9 [132.9, 151.1]	31.88 [31.36, 31.88]	0.996
write	64	252 431 [250 115, 252 712]	252.8 [252.6, 255.2]	63.83 [63.80, 63.83]	0.997
write	128	247 061 [239 034, 247 254]	492.5 [491.5, 502.3]	121.77 [121.44, 127.54]	0.951
write	256	253 743 [242 276, 255 085]	999.1 [956.0, 1 003.7]	254.67 [254.67, 254.86]	0.995

0.97 GiB/s matches the SLC ingest rate measured in Section 5. Writes saturate the buffer's intake, not the NAND parallelism that limits reads.

7.2 Little's Law as a consistency check

Little's Law [2] relates the mean number of requests in a system L, the throughput λ and the mean time in system W: $L = \lambda W$. In a *closed-loop* harness it holds *by construction*. There are exactly QD request slots, and each completion is resubmitted immediately, so $\lambda W = \mathrm{QD} \cdot W/(W + g)$, where g is the host-side gap between reaping a completion and resubmitting it. Matching QD therefore does not show that the device behaves in a particular way. It shows that the timers are consistent, that no slot is lost, and that the host keeps up ($g \ll W$). A ratio L/QD clearly below 1 would mean the issuing thread, not the device, is the bottleneck. On a tmpfs control, where I/O completes synchronously inside `io_submit`, `make test` measures $L/\mathrm{QD} \approx 0.57$ at QD=8. Note also that W is measured from submission, so L counts requests in the kernel block layer and driver as well as in the device; the true device-queue occupancy can only be lower.

Inference: internal parallelism. Little's Law can also be applied to the *device*, as an inference rather than a measurement. At saturation the drive completes $\lambda_{\max} \approx 159\,047$ reads/s. If an isolated 4 KiB read occupies the flash back end for roughly its unloaded service time S, the mean number of reads *in service* is $L_{\mathrm{busy}} = \lambda_{\max} S$. Taking S between 40 µs (the QD=1 latency minus ∼10 µs of host and transfer overhead) and 53 µs (the whole QD=1 latency) gives $L_{\mathrm{busy}} \approx 6$–$8$. So of the 64–256 requests outstanding past the knee, only about 6–8 are being served by the NAND at any moment, and the rest wait in queues. That is the queueing-delay component of the latency growth in Figure 4. Micron does not publish the die or channel count for the 2210. A 512 GB drive built from 1-Tbit QLC dies would have 4 dies, in which case 6–8 concurrent reads would mean about two reads in flight per die (multi-plane or independent-plane reads). This is a plausibility argument

with an assumed S, not a measured die count.

7.3 Analysis

Read IOPS climb from 18 866 at QD=1 to 96 303 at QD=16 and 129 513 at QD=32, then flatten at 149 422–159 047. Scaling is sub-linear from the start. Successive doublings from QD=1 add 56, 39, 60, 47, 34, 15, 4.5 and 1.8 %, with no distinct regime of linear scaling. The pilot run's impression that QD 1→4 scaled unusually poorly did not reproduce: here it scaled 2.2×. Beyond the knee, mean latency doubles with each doubling of QD (424→813→1 603 µs), which is the textbook saturation signature. With λ flat, $L = \lambda W$ can follow QD only if W grows in proportion, so every added request simply waits.

For reads, L/QD stays between 0.987 and 0.998 at every depth up to 256 (Table 6). The single `libaio` thread keeps the queue full, and the saturation belongs to the device. For writes the ratio is 0.95–0.997. The lowest value, 0.95 at QD=128, coincides with the widest p99.9 there (16 712 µs). We read it as completions bunching around a flush stall rather than a host limit, but with five repetitions we cannot separate the two.

The practical reading for reads: QD ≈ 32–64 extracts nearly all the throughput at a quarter to half a millisecond, and deeper queues only buy latency. Writes need only QD ≈ 16, as long as the SLC cache has room. Once it does not, write throughput is set by QLC programming, whatever the queue depth (Section 9).

8 Tail Latency

Hypothesis (from first principles, then compared with the data)

Moving from a mid queue depth to the near-knee depth should raise the median modestly but stretch the upper percentiles much more: deeper queues mean more requests waiting behind the occasional slow flash operation, so the p99/p99.9 gap over p50 should widen.

Fixed	4 KiB random read; one job; 6 s runs, 5 repetitions; NVMe temperature recorded per repetition
Varied	iodepth 8 (mid), 32 and 64 (bracketing the knee of Section 7), 128 (past it)
Script	`SSD_Performance_Profiling/scripts/run.sh tail`

8.1 Analysis

The tail runs did *not* see the same drive as the queue-depth read sweep, and that difference is the main result. In `run.sh all` they started immediately after the queue-depth *write* sweep, which wrote ∼11 GiB of random 4 KiB data, and there was no idle gap. At the same queue depth, the tail runs delivered half the IOPS of the earlier, clean read sweep: 64 475 vs. 129 513 at QD=32. Their median latency was 385 vs. 217 µs. The temperature was 50.9 °C to 52.9 °C, so this is not thermal. The likely cause is foreground reads competing with the controller's background folding and garbage collection of the data just written.

Under that contention the tail detaches from the body as QD grows, as hypothesised but more strongly:

- QD=8: p50 198, p99 623, p99.9 807 µs; p99.9/p50 = 4.1×.
- QD=32: p50 385, p99 1 812, p99.9 2 594 µs; 6.7×.
- QD=64: p99.9 7 614 µs; 15×.
- QD=128: p99.9 16 008 µs; 24×.

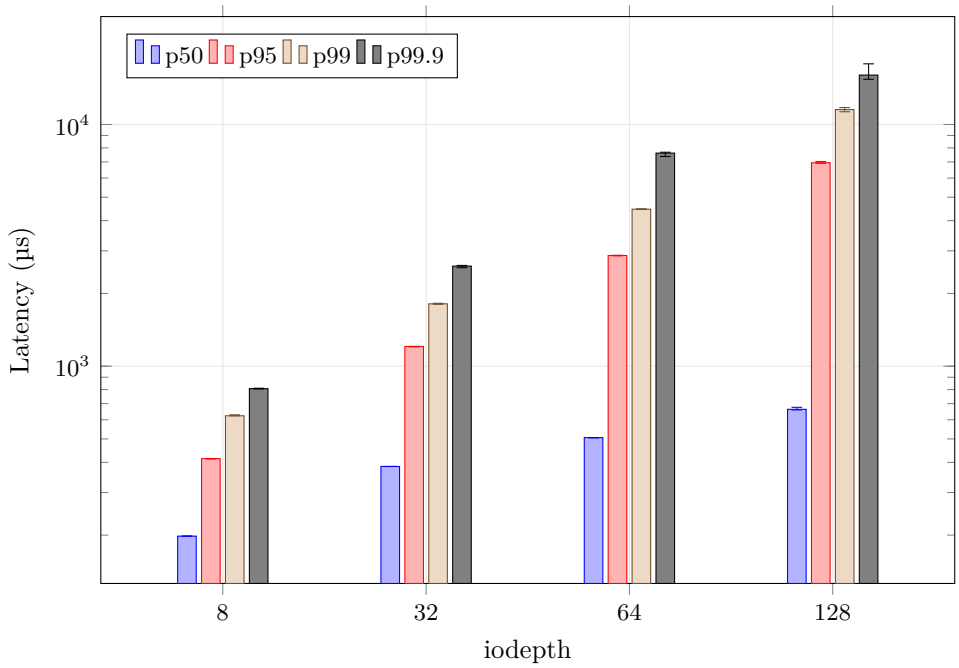


Figure 5: Latency percentiles at a mid queue depth (8), around the knee (32, 64) and past it (128); medians with IQR bars (data: 2026-10-21 run). In the 2026-10-21 run these points followed ∼11 GiB of random writes while the drive was still doing background flash work, and the upper tail detaches from the median as QD grows: p99.9/p50 rises from about 4× to 24×.

Meanwhile IOPS barely grow past QD=32 (64 475→72 985). In the clean sweep, by contrast, p99.9/p50 stays between 3.8 and 4.9× from QD=16 to 256: the whole distribution shifts up together, which is pure queueing. The tail ratio is therefore a sensitive detector of background flash activity. Queueing alone keeps it near 4×, and a read that arrives during a program/erase or a garbage-collection burst waits milliseconds. Deeper queues hold more requests behind each such stall.

These are the numbers that govern provisioning for a latency-SLA service, not the mean. At QD=32 on a drive that has recently absorbed writes, one read in a thousand takes 2 594 µs. The knee from Section 7 is therefore a latency decision as much as a throughput one: past it, almost no IOPS are gained and these tails inflate fastest. The NVMe composite temperature stayed at 33.9 °C to 61.9 °C across all runs, far below the 86.85 °C warning threshold (Section 9).

9 Device State and Working Set

Hypothesis (from first principles, then compared with the data)

Back-to-back sustained sequential writes should first ride the SLC/DRAM write cache (a burst), then settle to a lower steady-state throughput once the cache must flush to QLC in the background. Occasional latency spikes from garbage collection should appear in the upper tail even while mean throughput is steady.

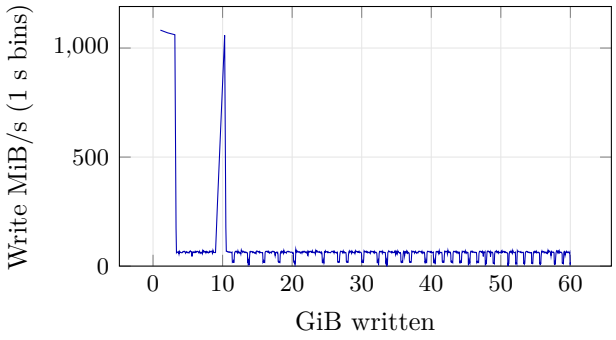
Table 7: Sustained-write summary (computed by `run.sh` into `sustained_summary.csv`). Burst is the median of the leading seconds that stay above half the first second's rate. The cliff is the first second, starting a 5-second window, whose rate falls below half the burst; the table gives the volume written before it (data: 2026-10-21 run). Steady is the median of all seconds after the cliff. A lean re-run stops the fill 10 seconds after the cliff; without a cliff the fill volume is a lower bound on the cache size. The random phase alternates between two states, so the table gives its overall average and the per-second quartiles.

Burst sequential write (MiB/s) / duration (s)	1 070 / 3
Cliff after (GiB written / s)	3.1 / 3
Post-cliff steady sequential write (MiB/s)	62
Steady / burst	0.058
Fill: GiB, seconds, average MiB/s	60.0, 1 228, 50
Random 4 KiB write, QD32: average IOPS	9 622
per-second IOPS quartiles; median per-second p99 (µs)	3 061–16 128; 32 253
Recovery after 60 s idle: first / last 5 s (MiB/s)	61 / 19
NVMe temperature min–max (°C)	45.9–55.9

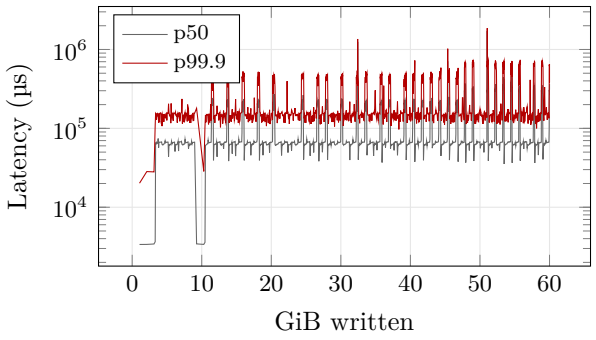
Hypothesis (from first principles, then compared with the data)

(Added before the large-file runs.) (i) A single sequential write far larger than the SLC cache should show a sharp *cliff*: burst throughput near the QD=32 write rate of Section 5, then a drop to the native QLC program rate, several times lower. (ii) After that, random 4 KiB writes reach a lower steady state than in the burst, with heavier tails from garbage collection. (iii) A short idle period lets the controller drain the cache and recover part of the burst rate. (iv) Random reads should slow as the working set grows from 1 GiB to 60 GiB, because more logical-to-physical map lookups miss the controller's mapping cache. (v) "Sequential" reads of data written randomly should be much slower than of data written sequentially. (vi) Temperature should rise during sustained writes but stay below the 86.85 °C warning threshold reported in sysfs (`temp1_max`).

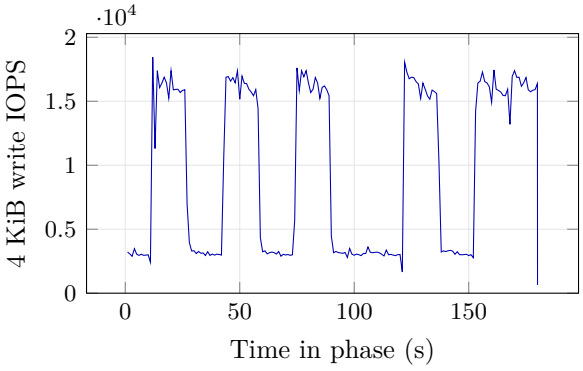
`sustained`	one O_DIRECT sequential pass over a fresh 60 GiB file (128 KiB, QD32; the file is `fallocate`d, so every byte is written exactly once). Then 4 KiB random writes at QD32 over the whole file for 180 s or 8 GiB, a 60 s idle gap, and a 15 s / 4 GiB sequential recovery burst. Throughput, p50/p99/p99.9/max latency and NVMe temperature are logged every second
`working_set`	4 KiB random read (QD1, QD32) and random write (QD32, 256 MiB cap) restricted to the first 1, 4, 16 and 60 GiB of the written file
`layout`	on the 2 GiB test file: 128 KiB sequential read (QD1, QD32) and 4 KiB random read, first after a sequential fill, then after a 4 GiB random 4 KiB overwrite (about 86 % of blocks rewritten in random order)
Volume / safety	`SUSTAIN_GIB=60` (the default 64 would have left less than the required 10 GiB free); the file is deleted afterwards. The filesystem has no online discard, so the freed blocks reach the SSD only at the weekly `fstrim`
Drive state	`device_state` started at 10:54, *immediately* after `run.sh all` (10:32–10:54, ∼60 GiB written), with no idle period and no TRIM (no root; `fstrim` runs weekly)
Script	`SUSTAIN_GIB=60 run.sh device_state` (= `sustained` then `working_set` on the same file); `run.sh layout` (inside `all`)



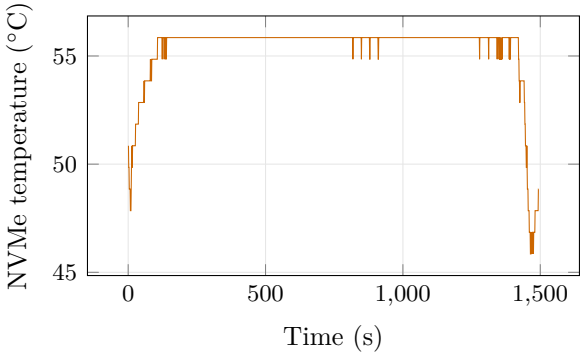
(a) Sequential fill: throughput vs. volume.



(b) Sequential fill: per-second latency.



(c) Random-write phase (QD32).



(d) Composite temperature, all phases.

Figure 6: Sustained-write experiment, one-second samples (data: 2026-10-21 run). (a) The SLC-cache cliff appears as the drop in throughput after the burst; (b) the latency tail during the fill; (c) random-write steady state after the fill; (d) temperature across the baseline idle, fill, random, idle and recovery phases.

9.1 SLC-cache exhaustion and steady state

Analysis. The fill began at the SLC ingest rate, 1 070 MiB/s, for only 3 s. After 3.1 GiB it fell to about 62 MiB/s and stayed there for the remaining 57 GiB: the median of all post-cliff seconds is 62 MiB/s, and the 20-minute average is 50 MiB/s. A burst rate 17× the steady rate is far more than the "several times" the hypothesis expected. The post-cliff rate is what this QLC drive sustains when it must program QLC directly while also folding cached data. It has two sub-states. Most 60-second windows sit at 60 MiB/s to 64 MiB/s. Eight of the 21 drop to 28 MiB/s to 46 MiB/s, with per-second p99 latencies of 0.3 s to 0.5 s at QD32. One isolated second at 97 s wrote 1.03 GiB at full SLC speed, presumably into a block of cache the controller had just freed.

What this run can and cannot say about the cache size. Only about 3 GiB of SLC was available when the fill started. That is a property of the run order, not of the drive. `device_state` began immediately after `run.sh all` had written ∼60 GiB in 22 minutes, with no idle time and no TRIM, so the cache was already nearly full. The fresh-drive burst-then-cliff transition, and hence the full dynamic SLC capacity, was therefore *not* captured, and we report no cache size. What the rest of the study shows is the burst *rate*, consistently: ∼1.06 GiB/s in the sequential-write sweep at QD=32 (Section 5), and 128 KiB QD=1 sequential writes at 8 132 IOPS (∼1.0 GiB/s) in `fio_check`, both from different experiments earlier in `all`. The block-size sweep also caught one exhaustion episode, after about 10 GiB of writes (Section 5). Measuring the true cache size needs a fill started on a drive that has idled and been trimmed.

The random-write phase averaged 9 622 IOPS (∼38 MiB/s). This is about 22× below the burst-state random-write rate at the same QD (209 717 IOPS, Section 7), and it alternated between two states. Per-second IOPS quartiles are 3 061 and 16 128: stretches near 16k IOPS, where writes still land in freshly freed SLC, alternate with stretches near 3k IOPS, where foreground writes wait for garbage collection. The median per-second p99 is 32 ms. After a 60 s idle there was *no* recovery. The recovery burst ran at 61 MiB/s for its first seconds and fell to 19 MiB/s by the end. With tens of gigabytes waiting to be folded into QLC, a minute of idle frees essentially no SLC, so hypothesis (iii) fails at this time scale.

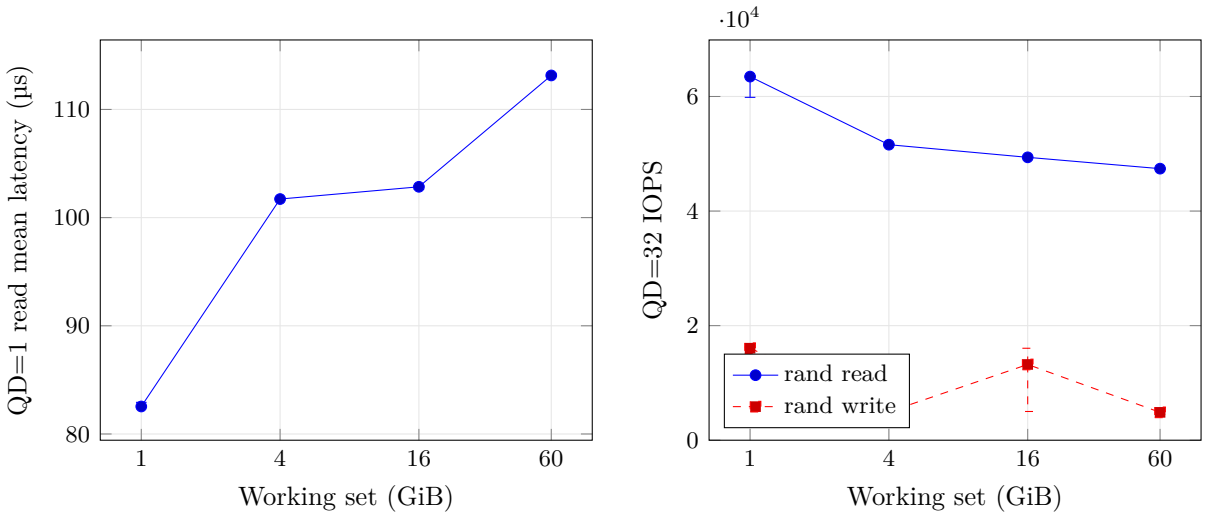
Two caveats. First, the file covers 60 GiB of a 512 GB drive, and the filesystem does not trim on delete. This is steady state for this region at this fill level, not the SNIA full-drive steady state, which would need the raw device (ruled out by the data-safety rule). Second, the drive started this run in a used state, as described above.

Temperature. The composite temperature ranged from 45.9 to 55.9 °C during this experiment, and from 33.9 °C to 61.9 °C over all runs (the peak was during the QD=32 sequential-read sweep). Both are far below the 86.85 °C warning and 89.85 °C critical thresholds reported by the drive. The throughput drop happened within seconds, at 50.9 °C, and did not track temperature afterwards (flat at 55.9 °C while throughput alternated), so it is the cache, not thermal throttling.

9.2 Working set (LBA range)

QD=1 random-read latency goes from 82.5 µs at 1 GiB to 101.7 (4), 102.8 (16) and 113.1 µs at 60 GiB, a rise of about 37 %. QD=32 read IOPS fall from 63 470 to 47 403 (−25 %). The direction fits logical-to-physical map lookups missing the controller's mapping cache more often as the span grows: at 4 bytes per 4 KiB page, 60 GiB needs 60 MiB of map. Most of the increase happens between 1 and 4 GiB, so the cached portion of the map is small. We could not tell whether the drive uses a host memory buffer (no allocation message in the kernel log).

Two confounds limit this conclusion. First, every working-set point ran right after the sustained



(a) QD=1 random read latency.

(b) QD=32 random read and write IOPS.

Figure 7: 4 KiB random I/O restricted to the first 1–60 GiB of the written file (medians, IQR bars; data: 2026-10-21 run). The runs followed the sustained write, so the drive was still folding in the background..

Table 8: The same reads on the same 2 GiB file after a sequential fill and after a random 4 KiB overwrite (medians of 5 repetitions).

Read	after sequential fill	after random overwrite
128 KiB seq, QD=1 (MiB/s)	1 443	224
128 KiB seq, QD=32 (MiB/s)	2 093	229
4 KiB rand, QD=1 (IOPS)	18 803	18 747

experiment, with tens of gigabytes still being folded. Even the 1 GiB span is far slower than the 53.1 µs QD=1 random read on the 2 GiB test file earlier, so absolute values describe a busy drive. Second, the spans ran in increasing order, so the trend is also confounded with elapsed time. Random writes (16 062, 4 963, 13 223 and 4 865 IOPS for spans of 1, 4, 16 and 60 GiB) show no span trend. They jump between the same two garbage-collection states as the random-write phase above, which supports the expectation that write cost does not depend on logical address once the cache is full.

9.3 Data layout: why preconditioning order matters

An early pilot run, since discarded, exposed this effect by accident: a 128 KiB QD=1 sequential read run right after a random-write test was several times slower than the block-size sweep. An SSD places data in the order it was *written*, not by logical address. Table 8 tests this directly, and the hypothesis is confirmed strongly. After the random overwrite, QD=1 sequential reads drop from 1 443 to 224 MiB/s (6.4×). 4 KiB random reads are unaffected (18 803 vs. 18 747 IOPS), since a single page costs the same wherever it sits.

Our sub-prediction that QD=32 would hide the gap was *wrong*. At QD=32 the fragmented file gives 229 MiB/s against 2 093, a 9.2× gap, and no better than at QD=1. Each fragmented 128 KiB request becomes up to 32 scattered page reads. The drive completes ∼1830 such requests per second (about 58k page reads/s) at either depth, well below the 130k random 4 KiB reads/s it sustains

Table 9: Same 4 KiB sequential-read workload, direct vs. buffered: medians with the IQR in brackets. Only the `O_DIRECT` flag differs.

Setup	IOPS	MiB/s	Mean (µs)	p99 (µs)
`O_DIRECT` (correct)	70 920	277	14.03	18.1
buffered (page cache)	1 334 404	5 213	0.68	1.15

at QD=32. We hypothesise that the controller serializes the fragments of one command, or that read-ahead actively wastes bandwidth on the wrong pages, but we did not test this.

This is the "insufficient preconditioning" pitfall in a subtler form. A sequential benchmark is meaningful only if the data's write history is controlled. That is why `run.sh` rewrites the test file sequentially before every sequential-read group. One group missed this in the run reported here: in `all`, `fio_check` followed `layout` without a rewrite, so its sequential points measured the fragmented layout (Section 3.1). The script has since been fixed.

10 Misleading Setup

Hypothesis (from first principles, then compared with the data)

Dropping `O_DIRECT` lets the OS page cache serve repeated reads of the same region from RAM after the first pass, so a buffered benchmark will report throughput and latency that describe DRAM, not the SSD—orders of magnitude "better" than the real device, and entirely fictitious as a storage result.

Pitfall	Buffered I/O measuring the page cache instead of the device
Wrong setup	4 KiB sequential reads over a 64 MiB region, *no* `O_DIRECT`
Right setup	Identical workload *with* `O_DIRECT` (page cache bypassed)
Fixed	QD=1, 5 repetitions of 4 s; test file rewritten sequentially just before
Script	`SSD_Performance_Profiling/scripts/run.sh pitfall`

10.1 Explanation

The buffered run reports 1 334 404 IOPS at 5 213 MiB/s and sub-microsecond latency, a 19× inflation over the correct 70 920 IOPS. None of it is a storage measurement. After the first pass the 64 MiB region lives entirely in the page cache, so every later read is a `memcpy` from RAM. The MiB/s figure is memory-copy bandwidth, and the latency is a cache hit, not an NVMe access.

This is the easiest way to publish an impossibly fast SSD number, and an AI-generated benchmark falls into it readily because omitting `O_DIRECT` is the *default* behaviour of `open()`. Three cheap detectors flag it:

1. The throughput exceeds the 3756 MiB/s PCIe 3.0 ×4 link ceiling measured from sysfs (Section 2). No device result can do that.

2. Sub-microsecond "storage" latency is physically impossible for NAND flash.

3. The number depends on the *region size*: it collapses once the region exceeds free RAM, which a true device result must not.

The methodology avoids the trap by construction. `O_DIRECT` is the default for every real run, and the buffered path exists only to produce this comparison. `make test` also asserts that buffered re-reads are more than 5× faster than direct ones on the scratch filesystem, which proves the cache is actually bypassed. Two related traps are covered elsewhere. A wrong effective queue depth is caught by L/QD in Table 6. Insufficient or wrong-order preconditioning, which cut a "sequential" read to a sixth of its true speed in our pilot, is covered in Section 9.3.

11 Anomalies and Limitations

11.1 Unexpected results

- **Random writes beat random reads, until the cache fills.** At QD=1, 4 KiB writes take 16.0 µs and reads 53.2 µs. At QD=16, pure writes (253 874 IOPS) beat pure reads (102 514). The controller acknowledges writes from its SLC/SRAM buffer. The sustained run confirms the mechanism: once the buffer is full, random writes average 9 622 IOPS, far below reads.
- **Write collapses inside ordinary sweeps.** Three block-size random-write points (≥256 KiB) and the QD=8 queue-depth write point fell to the direct-to-QLC rate, with p99 of tens of milliseconds, while their neighbours did not. Hypothesis: transient SLC exhaustion after several GiB of writes. These points are reported, not dropped, and the knee rule's output for writes is overridden in the text.
- **Sequential-read speed depends on write history** (6.4× at QD=1 and 9.2× at QD=32; Section 9.3). Deep queues did not hide the gap, contrary to our prediction.
- **Tails depend on recent writes.** At the same QD, tail runs right after the queue-depth write sweep had half the IOPS and up to 5× larger p99.9/p50 ratios than the clean sweep (Section 8).
- **FIO and `blkbench` differ on QD=32 random writes** (0.78), with matching medians and a heavier FIO tail. Hypothesis: FIO always ran second in each pair, after `blkbench` had just written 256 MiB (Section 3.1). Not tested.
- **No recovery after** 60 s **idle** following the sustained write. The fold backlog needs much longer.

11.2 Limitations

- **Run order shaped the device-state results.** `device_state` ran immediately after `all`, with no idle and no TRIM. Only ∼3 GiB of SLC was free, so the fresh-drive SLC capacity was not measured, and the working-set numbers describe a drive busy with background folding. A clean measurement needs an idle and trimmed drive. That needs root for `fstrim`, or waiting for the weekly timer.
- **Primary engine is not FIO.** The sweeps use `blkbench`. FIO cross-checks it on seven points (Section 3.1): reads agree within 2 %, QD=32 random writes do not. `ENGINE=fio` reruns every sweep through FIO.
- **Closed-loop load only.** Every experiment holds a fixed QD. An open-loop (fixed arrival rate) generator would show queueing delay against offered load, and its tails would differ.

- **No `iostat/perf` cross-check.** Effective queue depth is checked in-band through L/QD (Table 6). That includes time in the kernel block layer, so it bounds device-queue occupancy from above.
- **SMART needs root.** Temperature comes from the NVMe hwmon sensor, which needs no root. Data-units-written, media errors and the throttle counters are unavailable, so throttling is inferred only from temperature and throughput together.
- **Filesystem, not raw device; partial LBA range.** I/O goes to disposable files on ext4: a 2 GiB file for the sweeps and a 60 GiB file for device state, about 13 % of the drive. Full-drive SNIA steady state is out of scope.
- **Byte caps and few repetitions on write points.** Write points stop after 256 MiB per repetition and use 3–5 repetitions, so a single flush stall can move a median (QD=8 writes). The QD=32 FIO comparison suggests a ∼20 % drive-state uncertainty on write numbers.
- **One `fio_check` group was not re-filled.** Its sequential points measured the fragmented layout left by `layout`. This does not affect the engine comparison, and the script has since been fixed.
- **Fixed data pattern.** All writes use an incompressible pattern. Compressibility was not varied (`blkbench -fill zero` exists for that). One drive, one firmware (`P6MA001`).
- **Report build backend.** The host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`.

12 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code for coding, debugging, analysing the data and drafting this report. Claude Opus 5 did the initial harness and report, up to 2026-10-20. Claude Opus 5.5 did the 2026-10-21 rework and the analysis of the final runs. GCC gcc (GCC) 16.2.1 20260810 built the code. The AI wrote the code and the prose. The timed runs were executed by the author in a terminal on the machine in Table 1, using the scripts Claude Code wrote (`run.sh all`, then `SUSTAIN_GIB=60 run.sh device_state`). Claude Code ran no timed benchmark; every number comes from those runs.

What they helped generate

`blkbench.c` (the O_DIRECT `libaio` benchmark), the data-safety gate (`safety.sh`) and its tests (`validate.sh`), the experiment driver (`run.sh`) with its CSV reduction, the FIO job files and converter, and the first drafts of these sections.

What was independently modified, tested, or rejected

Concrete corrections:

1. *The wrong device was being tested.* The scratch directory defaulted to the in-repo `data/scratch/`, which lives on the 2 TB SATA *HDD*, so the first full dataset showed millisecond QD=1 latencies. The scratch path was made overridable (`ACS_SCRATCH`) and pointed at the NVMe SSD. The AI's claim that "the NVMe is read-only to us" was

false, as a one-line write test showed. The device check in `env.txt` was itself buggy: it stripped trailing digits from `nvme0n1p3` and printed `rotational: ?`. It now resolves the partition to its parent disk.

2. An early version measured *submission* latency rather than *completion* latency; it now times each I/O to completion.
3. A convenience default left the page cache enabled and reported ∼2 million IOPS. That case became the required pitfall (Section 10), and `O_DIRECT` became the default.
4. The report claimed a `0xA5` write fill, but the timed write buffers were never initialised. They are now filled once with an incompressible xorshift pattern, and `make test` checks that it does not compress.
5. The report claimed `validate.sh` refused `/dev/nvme0n1` when it only tested `/dev/sda`; it now tests both. It also claimed a direct-vs-buffered assertion that did not exist; the assertion has been added.
6. An AI-drafted sentence said p99 "rises monotonically with write fraction". The table contradicts it (pure write is below 50/50), and the sentence was corrected.
7. The first version of the sustained-write summary defined "burst" as the median of the first 10 s. On the real data, where the burst lasted only 3 s, that reported a misleading 66 MiB/s burst and a spurious cliff at 11 GiB. The definitions were fixed and the summary was regenerated from the unchanged raw log (`run.sh reprocess`).
8. Claims carried over from the pilot data were re-checked against the final run and corrected where the data disagreed: the QD 1→4 scaling, the 26 % gain from QD 32 to 64 (15 %), the constant p99.9/p50 ratio, and the prediction that deep queues hide fragmentation.

Validation

`make test` checks the safety gate (both block devices, a symlink to one, `/etc/passwd`, `/`), the incompressible fill, physical orderings, the $L \leq \mathrm{QD}$ invariant, page-cache bypass and the FIO converter. FIO cross-checks `blkbench` (Section 3.1). Every figure traces to a committed raw/processed file (Section A).

13 Conclusion

The Micron 2210 NVMe SSD shows the queueing-driven throughput–latency tradeoff the project set out to measure. At QD=1, 4 KiB random reads cost 53.2 µs and writes only 16.0 µs, because the controller's write buffer hides the flash program.

Block size moves the bottleneck from per-request latency to bandwidth. With one request in flight, sequential reads stay near 1.4 GiB/s from 64 to 256 KiB and reach 1 798 MiB/s at 1 MiB. At QD=32 they top out at 2 144 MiB/s, 57 % of the 3756 MiB/s PCIe 3.0 ×4 link. The NAND back end, not the interface, is the ceiling. Writes saturate at ∼1.06 GiB/s of SLC ingest even at QD=1. Mixing reads with writes is worse than either pure workload.

The read knee is at QD 64 under an explicit 10 %-per-doubling rule. Beyond it, latency doubles with every doubling of QD for at most a few percent more IOPS. L/QD stays at 0.987–0.998. Little's Law holds by construction in a closed loop, so this confirms that the harness keeps the queue full and the saturation is the device's.

Device state matters as much as configuration. Once the SLC cache is full, sequential writes fall from 1 070 to 62 MiB/s (17×), and random writes average 9 622 IOPS with 32 ms per-second p99. A minute of idle does not bring the cache back. Reads issued while the drive is still folding recent writes lose half their IOPS, and their p99.9 grows up to 24× the median. Sequential reads of randomly written data run 6–9× slower whatever the queue depth. Because our device-state run

started on a cache already filled by the preceding sweeps, the fresh-drive SLC capacity remains unmeasured. The NVMe temperature peaked at 61.9 °C, far below the drive's 86.85 °C warning threshold, so we saw no thermal effects.

The sharpest lessons are methodological. Without `O_DIRECT`, the same workload reported page-cache fiction. The first dataset was taken on the HDD until the `rotational` flag gave it away. A "sequential" read can be six to nine times slower depending only on how the data was written. And on a QLC drive, *what ran just before* changes write throughput by an order of magnitude and read tails severalfold. A storage measurement can be trusted only if it bypasses the cache, verifies its effective queue depth, controls the data's write history and the drive's recent workload, and confirms which device it is talking to.

References

[1] J. Axboe, *Fio: Flexible I/O tester*, Version 3.42 on the test host; used to cross-check the libaio benchmark (blkbench). [Online]. Available: `https://github.com/axboe/fio`

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

A Reproducibility

Code, scripts, raw data and this report live in one repository: `ACS-Code/` holds the code and data, and `ACS-Latex/` holds the reports. The exact commit is `b00db669940b`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout b00db669940b
cd ACS-Code
make tools                                    # build/tools/summarize
make -C SSD_Performance_Profiling             # build/blkbench (O_DIRECT, libaio)
make -C SSD_Performance_Profiling test        # safety + sanity checks (~70 MiB written, deleted)
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
export ACS_SCRATCH="$HOME/acs_ssd_scratch"    # a directory on the SSD under test
SSD_Performance_Profiling/scripts/run.sh dryrun        # device, link, temperature, space; writes
    nothing
SSD_Performance_Profiling/scripts/run.sh all           # 2 GiB-file sweeps + fio_check (~23 min
    measured)
SUSTAIN_GIB=60 SSD_Performance_Profiling/scripts/run.sh device_state  # sustained + working set (
    needs 70 GiB free)
scripts/sync_data.sh SSD_Performance_Profiling
cd ../ACS-Latex && make ssd
```

Clean re-run. Several results in this report are shaped by the run order: `all` followed directly by `device_state`, with no TRIM and no idle (Section 2). `scripts/rerun_clean.sh` repeats the affected experiments in an order that avoids these confounds. It first checks free space (at least `SUSTAIN_GIB`+10 GiB), drive idleness (`/sys/block/nvme0n1/stat` over 10 s) and recent-TRIM evidence in the journal. It then runs:

1. the 60 GiB sustained fill on the fresh drive;
2. a 20 min idle, then the working set with spans forward and reverse;
3. a 10 min idle, then tail on a freshly preconditioned file;
4. zero-queue;

5. block-size, now with QD=32 random reads and a 15 s pause before each write point;
6. queue-depth;
7. `fio_check` with alternating engine order.

Steps 5–7 each follow a 5 min rest. The upper bound is ∼135 GiB written in ∼2 h; peak space in use is 60 GiB.

```
sudo fstrim -v /                # then leave the machine idle >= 30 min
export ACS_SCRATCH="$HOME/acs_ssd_scratch"
SSD_Performance_Profiling/scripts/rerun_clean.sh dryrun
SSD_Performance_Profiling/scripts/rerun_clean.sh
scripts/sync_data.sh SSD_Performance_Profiling
```

Lean re-run. `rerun_clean.sh lean` is a shorter version of the clean re-run: ∼35 GiB written expected, 55 GiB at most, in about 35 min to 50 min. Its steps are:

1. The sustained fill, first, that *stops at the cliff*: 10 seconds after throughput stays below 30 % of the burst median, capped at 40 GiB. With no cliff by then, the result is reported as a lower bound on the cache size. A short random phase, a 60 s idle and a recovery probe follow, and the large file is deleted.
2. A 10 min rest, then tail on a freshly preconditioned file.
3. The working set over 1, 2 and 4 GiB, forward and reverse.
4. QD=32 random reads, then the ≥256 KiB random writes again after a rest.
5. Write queue depths 4–32.
6. FIO write points with alternating engine order.

It also takes an NVMe SMART snapshot before and after, when `nvme smart-log` works without root. Partial re-measurements replace only the matching configurations of the 2026-10-21 run, and figure captions say which run each point comes from.

```
sudo fstrim -v /                # then leave the machine idle >= 30 min
SSD_Performance_Profiling/scripts/rerun_clean.sh lean dryrun
SSD_Performance_Profiling/scripts/rerun_clean.sh lean
```

`ACS_SCRATCH` selects the device under test, and the `safety.sh` gate is identical wherever it points: a plain disposable file inside the scratch directory, never a device or mountpoint. Without `ACS_SCRATCH`, the in-repo `data/scratch/` filesystem is profiled instead (an HDD on the test host). Knobs (`REPS`, `RUNTIME`, `WRITE_CAP_MIB`, `SUSTAIN_GIB`, `ENGINE=fio`, ...) are documented at the top of `run.sh`. `run.sh reprocess` rebuilds every processed CSV from the newest raw run of each experiment.

The timed runs reported here were executed by the author in a terminal on the test machine, with the CPU governor set to `performance`, turbo off and SMT off. `run.sh all` started at 10:32 and `SUSTAIN_GIB=60 run.sh device_state` followed immediately at 10:54 on 2026-10-21.

Each raw directory holds `raw.csv` (one row per repetition or per second, with full configuration metadata and the NVMe temperature) and `env.txt`. `env.txt` contains the `collect_env.sh` snapshot plus the scratch filesystem's parent disk, its `rotational` flag, model, firmware, PCIe link, scheduler, temperature and thresholds, and the knob values. FIO runs also keep each JSON result under `fio/`. `tools/summarize` reduces the raw files to the processed CSVs plotted here, and `scripts/sync_data.sh` copies them into this report's `data/`. Tables and quoted numbers are read from those CSVs when the report is compiled, so every number traces to one of these files.

Table 10: Raw data behind each figure (`ACS-Code/SSD_Performance_Profiling/data/raw/...`).

Figure / table	Raw run directory
Table 4	`zero_queue/20261021-103201`
Figures 1 and 2	`block_size/20261021-103201`
Figure 3, Table 5	`rw_mix/20261021-103201`
Figure 4, Table 6	`queue_depth/20261021-103201`
Figure 5	`tail/20261021-103201`
Table 9	`pitfall/20261021-103201`
Table 8	`layout/20261021-103201`
Table 3	`fio_check/20261021-103201` (FIO JSON under `fio/`)
Figure 6, Table 7	`sustained/20261021-105439`
Figure 7	`working_set/20261021-105439`